Deep Learning:

A Deep Learning Model To Recognize Emotion At Utterance Level In Conversation Using PyTorch

By:

Jemish Mavani (jemishmavani@outlook.com)

CONTENTS

1. **INTRODUCTION**
2. **AIM**
3. **MOTIVATION**
4. **APPROACH**
5. **DATASET**
6. **TRAINING**
7. **OBSERVATION OF RESULT**
8. **CONCLUSION**
9. **REFERENCES**

1. INTRODUCTION

This section briefly describes Deep Learning and Emotion Recognition.

What is Deep Learning?

It is a sub part of machine learning method that takes in an input X, and uses it to predict an output of Y. Given a large dataset of input and output pairs, a deep learning algorithm will try to minimize the difference between its prediction and expected output. By doing this, it tries to learn the association/pattern between given inputs and outputs — this in turn allows a deep learning model to generalize to inputs that it hasn't seen before. The "deep" part of deep learning refers to creating deep neural networks. This refers a neural network with a large number of layers — with the addition of more weights and biases, the neural network improves its ability to approximate more complex functions.

What is Emotion Recognition?

Simply put, emotion recognition (ERC) is the task of classifying the emotion behind a piece of written task. Given a piece of text can you tell if the speaker is angry, happy, sad, or perhaps confused? It has many far-ranging applications in healthcare, education, sales, and human resources. At the highest level, the task of ERC is useful because many are convinced that it is a stepping stone to building a conversationally intelligent AI that is able to talk with a human.

An example of using ERC to power a health assistant,

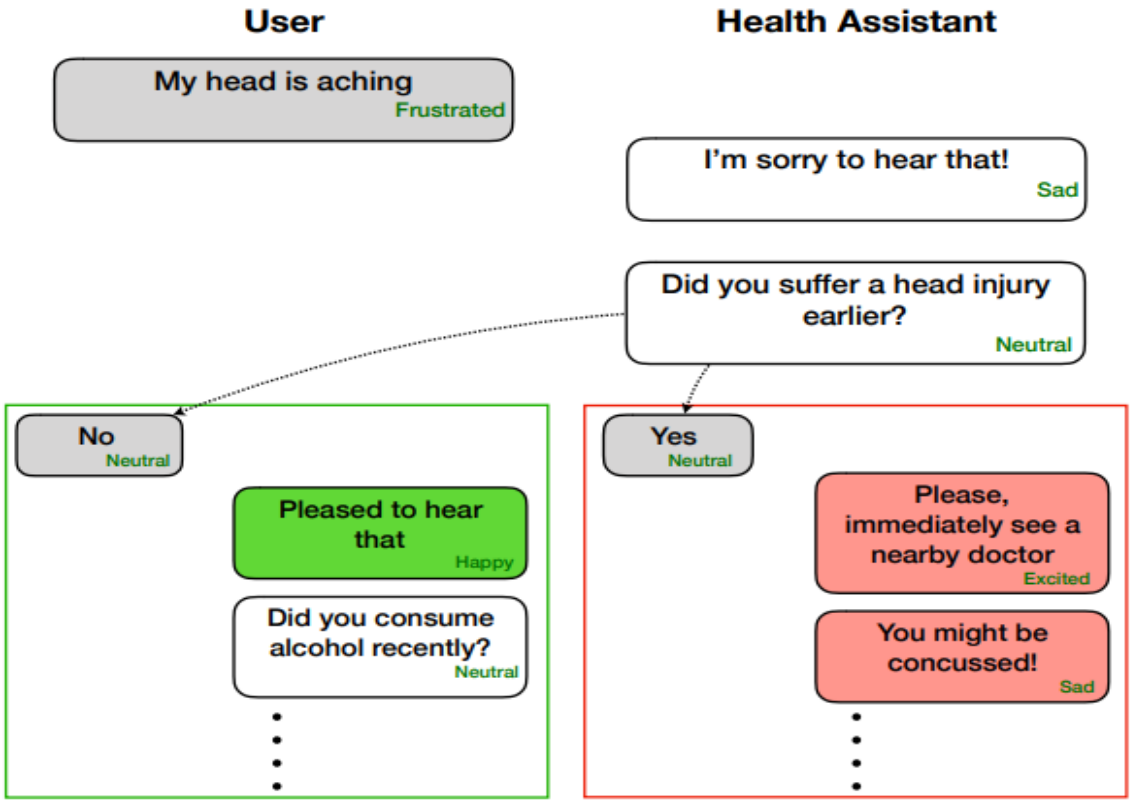


Image taken from [1]

2. AIM

To create an RNN and Attention based deep learning model for recognizing emotions at utterance level in conversation happening between speakers.

3. MOTIVATION

The two major pieces of innovation that most Emotion Recognition (ERC) is built on are recurrent neural networks (RNN) and attention mechanisms. RNNs such as LSTMs and GRUs look at text sequentially. When the text is long, the model's memory of the beginning parts is lost. Attention mechanisms solve this well by weighing different parts of the sentence differently.

However, RNNs+Attention still have trouble taking into context of personality, topic, and intent from neighboring sequences and also the speaker (basically all the very important parts of any conversation). Couple this with the lack of labeled benchmark datasets for personality/emotion it becomes really difficult not just to implement but also to measure the result of new models. So, a relatively new innovation proposed to solve the issue called graph convolution networks: DialogueGCN: A Graph Convolutional Neural Network for Emotion Recognition in Conversation [1].

4. APPROACH

I followed the same approach as DialogueGCN model and trained it. This section describes the components of DialogueGCN and its approach to recognize the emotions.

In a conversation, context matters. A simple “Okay” can mean “Okay?”, “Okay!” or “Okay….” depending on what you and the other said before, how you are feeling, how the other person is feeling, the level of tension and many other things. There are two types of context that matter:

1. Sequential Context: The meaning of a sentence in a sequence. This context deals with how past words impact future words, the relationship of words, and semantic/syntactic features. Sequential context is taken into account in RNN models with attention.

2. Speaker Level Context: Inter and intra-speaker dependency. This context deals with the relation between the speakers as well as self-dependency: the notion that your own personality change and affect how you speak during the course of the conversation.

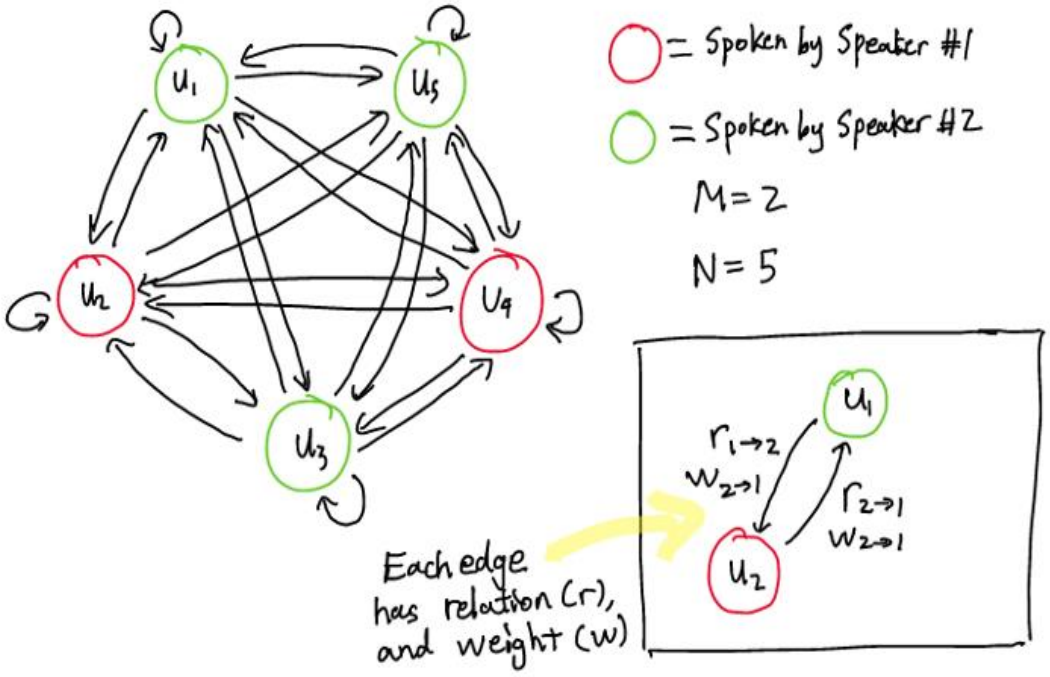
it’s speaker level context that most models have difficulty taking into account. It turns out that modelling speaker level context can be done using graph convolutional neural networks and this is exactly the approach DialogueGCN takes.

Representing Conversations as Graphs

In a conversation there are M speakers/parties represented as p[1], p[2], . . ., p[M]. Each utterance (a piece of text someone sends) is represented as u[1],

u[2], . . ., u[N]. The final goal in ERC is to accurately predict each utterance as one of happy, sad, neutral, angry, excited, frustrated, disgust, or fear.

The entire conversation can be built as a directed graph:



A conversation graph with 2 speakers and 5 utterances.

G = (V, E, R, W)

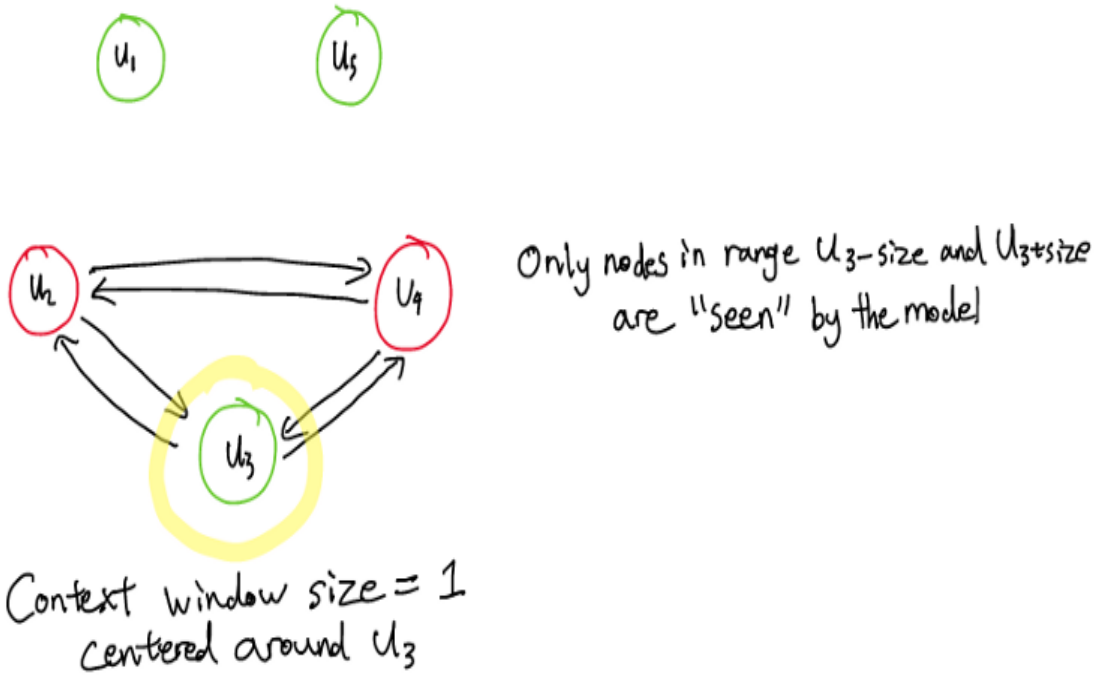
- Utterances are the nodes (V). Edges are paths/connections between the nodes (E). Relations are the different types/labels of the edges (R). Weights represent the importance of an edge (W).
- Every edge between two nodes v[i] and v[j] has two properties: the relation (r) and weight (w). We'll talk more about this in just a bit.

- The graph is directed. Therefore, all edges are one way. The edge from v[i] to v[j] is different from the edge that goes from v[j] to v[i].

Context Window

One major problem of a graph representation is that if a conversation is really long, there can be many edges for a single node. And since each node is connected to every other node, it scales quadratically as the size of the graph increases. This is very computationally expensive.

To resolve this problem, in DialogueGCN, the graph's edges are constructed based on a context window with a specific size. So for a node/utterance i, only the past size and future size utterances are connected in a graph (any node in the range of i-size to i+size).



A context window of size 1 around the third utterance

Edge Weights

Edge weight are calculated using an attention function. The attention function is set up so that for each node/utterance, the incoming edge weights all sum up to 1. Edge weights are constant and do not change in the learning process.

In simplified terms, the edge weight represents the importance of the connection between two nodes.

Relations

The relation of an edge depends on two things: Speaker dependency: Who spoke u[i]? Who spoke v[j]? Temporal dependency: Was u[i] uttered before u[j], or the other way around?

In a conversation, if there are M different speakers, there will be a maximum of M (speaker of u[j]) * M (speaker of u[j]) * 2 (whether u[i] occurs before u[j], or the reverse) = $2M^2$ relations.

At a high level, relation is an important property for the edge because who spoke what and when matters a lot in a conversation. If Peter asks a question and Jenny responds, this is different from Jenny first saying the answer and then Peter asking the question (temporal dependency). Likewise, if Peter asks the same question to Jenny and Bob, they may respond differently (speaker dependency).

Think of the relation as defining the type of the connection, and the edge weight representing the importance of the connection.

The Model

The DialogueGCN model uses a type of graph neural network known as a graph convolutional network (GCN). The model is composed of 3 parts:

1. Sequential Context Encoding
2. Speaker-Level Context Encoding
3. Classification

The example shown is for a 2 speaker 5 utterance graph:

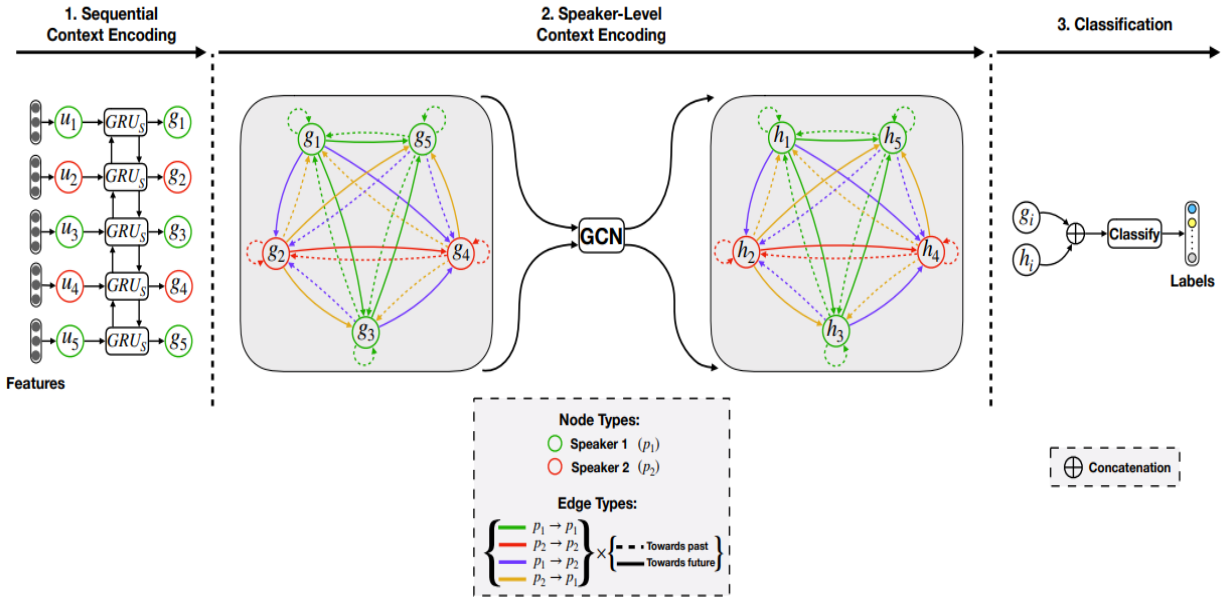


Figure 3 from [1]

In stage 1, each utterance u[i] is represented as a feature vector and given sequential context encoding. This is done by running each utterance through a series of GRUs in a sequential context encoder. A graph structure is not needed for stage 1. The new utterance with sequential context is denoted as g[1] ……. g[N] in the paper. This is the input to the GCN.

In stage 2, the model constructs a graph like discussed in the previous section and will add speaker level context to the graph using feature transformation. The utterances with BOTH sequential and speaker level context is denoted by h[1] ……h[N]. This is the output of the GCN.

In stage 3, the original sequential context encoded vectors are concatenated with the speaker level context encoded vectors. This is similar to combining original layers with later layers to “summarize” the outputs of every layer.

The concatenated feature vector is then fed into a fully connected network for classification. The final output is a probability distribution of the different emotions the model thinks the utterance is.

5. DATASET

DailyDailogue dataset is used for the training and testing. The features of dataset is as follows:

1. The language in DailyDialog is human written and thus is more formal than other datasets.
2. The conversations in DailyDialog often focus on a certain topic and under a certain physical context. For example, a conversation happens in a shop is often between a customer looking for suitable goods and a salesman who is willing to help for purchasing. Another typical conversation happens between two students talking about their summer vacation trips.
3. The third desirable feature is that the crawled dialogues usually end after reasonable speaker turns. This makes DailyDialog distinguished from existing dialog datasets.

DailyDialogue Statistics:

Total Dialogues: 13,118

Average Speaker Turns Per Dialogue: 7.9

Average Tokens Per Dialogue: 114.7

Average Tokens Per Utterances: 14.6

Emotions: Anger, Disgust, Fear, Happiness, Sadness, Surprise, No-Emotion

Overall, DailyDialogue is a good dataset to start with. Out of 13,118 dialogues, 11,118 is used for training, 1000 for validation and 1000 for testing.

6. TRAINING

DialogueGCN model is trained with the following parameters:

- Base-Model: GRU
- Past-Window Size: 7
- Future-Window Size: 7
- Class-Weights: True
- Active-Listener: False
- Nodal-Attention: False
- Attention-Type: General
- Learning-Rate: 0.003
- L2 Regularization: 0.0001
- Batch-Size: 32
- Dropout: 0.3
- Epoch: 10+20

The training of the model is done using a categorical-cross-entropy loss with L2-regularization. This loss is used because the model is predicting the probability of multiple labels (emotion classes). Due to limited GPU hours on Google Colab, I trained in two parts, first time 10 epochs and second time 20 epochs, **due to training on less epochs, prediction may not be the same all the time.**

7. OBSERVATION OF RESULT

For my training, I observed up to 87% accuracy on training dataset and for validation dataset I observed around 84% during 6th epoch of second time.

```
Epoch: 4, Time: 6m 5.4359s
train_loss: 0.3711, train_acc: 84.61, train_fscore: 85.08, train_precision: 84.61, train_recall: 85.56
valid_loss: 0.3897, valid_acc: 84.6, valid_fscore: 85.16, valid_precision: 84.6, valid_recall: 85.73
Model saved at ./checkpoints/DailyDialog/GraphGRU.pt...
================================================================================================

Epoch: 5, Time: 6m 2.9087s
train_loss: 0.3563, train_acc: 85.05, train_fscore: 85.53, train_precision: 85.05, train_recall: 86.01
valid_loss: 0.3806, valid_acc: 83.82, valid_fscore: 84.38, valid_precision: 83.82, valid_recall: 84.94
================================================================================================

Epoch: 6, Time: 6m 1.3030s
train_loss: 0.3466, train_acc: 85.47, train_fscore: 85.95, train_precision: 85.47, train_recall: 86.43
valid_loss: 0.4029, valid_acc: 84.86, valid_fscore: 85.42, valid_precision: 84.86, valid_recall: 85.99
Model saved at ./checkpoints/DailyDialog/GraphGRU.pt...
================================================================================================
```

Here's the training and validation statistics:

```
F-Score(Tr/Va): (85.95, 85.42)
Precision(Tr/Va): (85.47, 84.86)
Recall(Tr/Va): (86.43, 85.99)
```

On testing data, I observed the same accuracy as validation,

```
[3]  1 !python inference_DailyDialogue.py --inference-file 'dailydialog/test.json' --model-weights 'checkpoints/DailyDialog/GraphGRU.pt' --statistics

Using TensorFlow backend.
Inference on GPU...
0 -> happiness
1 -> no_emotion
2 -> disgust
3 -> surprise
4 -> sadness
5 -> fear
6 -> anger

Average Accuracy: 84.87
Average Fscore: 84.87
Precision: 84.87
Recall: 84.87
```

Here we get all the 4 value same but that is not the case all time.

During testing, I observed all the 4 values accuracy, fscore, precision and recall same but, that is not the case all the time.

```
Sentence: sounds good let ' s see i want
Actual Emotion: no_emotion
Predicted Emotion: surprise

Sentence: yeah
Actual Emotion: fear
Predicted Emotion: fear

Sentence: i want you to put your hands behind your head you are under arrest
Actual Emotion: no_emotion
Predicted Emotion: fear
```

8. CONCLUSION

The report presents a trained model using Deep Learning that recognizes the emotion in conversation between speakers with 7 emotion classes like Anger, Disgust, Fear, Happiness, Sadness, Surprise, No-Emotion, implemented in PyTorch.

Total 11,118 dialogues were used for training, 1000 for validation and 1000 were used for testing.

The training accuracy obtained was 87.23%, validation accuracy obtained was 84% and testing accuracy obtained was 84% as well. **HOWEVER, IF ENOUGH RESOURCES IS PROVIDED**, the model can be further enhanced and can be made more robust by training it for more epochs.

9. REFERENCES

I. [1] Ghosal, Deepanway & Majumder, Navonil & Poria, Soujanya & Chhaya, Niyati & Gelbukh, Alexander. (2019). DialogueGCN: A Graph Convolutional Neural Network for Emotion Recognition in Conversation.

II. https://towardsdatascience.com/emotion-recognition-using-graph-convolutional-networks-9f22f04b244e